

WEEK-8 SKILL

1 . To Detect Variable References, Expand Values, Handle Undefined Variables, Update Tokens, Support Nested Variables, Test Expansion Logic

Aim : To implement variable expansion in a shell-like environment by detecting variable references, replacing them with their values, handling undefined variables, updating tokens, supporting nested variables, and testing the expansion logic.

Program :

```
GNU nano 7.2
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>

#define MAX_OUTPUT 4096

void expand_variables(const char *input, char *output)
{
    int i = 0;
    int j = 0;

    while (input[i] != '\0' && j < MAX_OUTPUT - 1)
    {
        if (input[i] == '$')
        {
            i++;

            /* Handle ${VARIABLE} format */
            if (input[i] == '{')
            {
                i++;

                char variable[100];
                int k = 0;

                while (input[i] != '\0' &&
                    input[i] != '}' &&
                    k < 99)
                {
                    variable[k++] = input[i++];
                }

                variable[k] = '\0';

                if (input[i] == '}')
                    i++;

                char *value = getenv(variable);

                if (value != NULL)
                {
                    for (int x = 0;
                        value[x] != '\0' &&
                        j < MAX_OUTPUT - 1;
                        x++)
                    {
```

```
        output[j++] = value[x];
    }
    else
    {
        printf("Undefined variable: %s\n", variable);
    }
}

/* Handle $VARIABLE format */
else if (isalpha((unsigned char)input[i]) ||
        input[i] == '_')
{
    char variable[100];
    int k = 0;

    while ((isalnum((unsigned char)input[i]) ||
            input[i] == '_') &&
            k < 99)
    {
        variable[k++] = input[i++];
    }

    variable[k] = '\0';

    char *value = getenv(variable);

    if (value != NULL)
    {
        for (int x = 0;
             value[x] != '\0' &&
             j < MAX_OUTPUT - 1;
             x++)
        {
            output[j++] = value[x];
        }
    }
    else
    {
        printf("Undefined variable: %s\n", variable);
    }
}
else
{
    output[j++] = '$';
}
}
```

GNU nano 7.2

```
strcpy(input, "$NAME is studying $COURSE in $YEAR");  
expand_variables(input, output);
```

```
printf("Input : %s\n", input);  
printf("Output: %s\n\n", output);
```

```
printf("Test 3: Undefined variable\n");
```

```
strcpy(input, "Value of $UNKNOWN");  
expand_variables(input, output);
```

```
printf("Input : %s\n", input);  
printf("Output: %s\n\n", output);
```

```
printf("Test 4: Braced variable\n");
```

```
strcpy(input, "Student: ${NAME}");  
expand_variables(input, output);
```

```
printf("Input : %s\n", input);  
printf("Output: %s\n\n", output);
```

```
printf("Test 5: Nested variable\n");
```

```
setenv("MESSAGE", "Welcome $NAME", 1);
```

```
strcpy(input, "$MESSAGE");  
expand_variables(input, output);
```

```
printf("Input : %s\n", input);  
printf("First expansion: %s\n", output);
```

```
char nested_output[MAX_OUTPUT];
```

```
expand_variables(output, nested_output);
```

```
printf("Nested expansion: %s\n\n", nested_output);
```

```
printf("==== EXPANSION TEST COMPLETED ==== \n");
```

```
return 0;
```

```
}|
```

Output :

```
jhanasri@LAPTOP-1RN1M8E3:~$ mkdir variable_expansion
jhanasri@LAPTOP-1RN1M8E3:~$ cd variable_expansion
jhanasri@LAPTOP-1RN1M8E3:~/variable_expansion$ pwd
/home/jhanasri/variable_expansion
jhanasri@LAPTOP-1RN1M8E3:~/variable_expansion$ NAME="Jhanasri"
jhanasri@LAPTOP-1RN1M8E3:~/variable_expansion$ COURSE="OSSP"
jhanasri@LAPTOP-1RN1M8E3:~/variable_expansion$ YEAR="2026"
jhanasri@LAPTOP-1RN1M8E3:~/variable_expansion$ echo $NAME
Jhanasri
jhanasri@LAPTOP-1RN1M8E3:~/variable_expansion$ echo $COURSE
OSSP
jhanasri@LAPTOP-1RN1M8E3:~/variable_expansion$ echo $YEAR
2026
jhanasri@LAPTOP-1RN1M8E3:~/variable_expansion$ echo "Student: $NAME"
Student: Jhanasri
jhanasri@LAPTOP-1RN1M8E3:~/variable_expansion$ echo "$NAME is studying $COURSE in $YEAR"
Jhanasri is studying OSSP in 2026
jhanasri@LAPTOP-1RN1M8E3:~/variable_expansion$ echo $UNKNOWN
jhanasri@LAPTOP-1RN1M8E3:~/variable_expansion$ echo "Value = [$UNKNOWN]"
Value = []
jhanasri@LAPTOP-1RN1M8E3:~/variable_expansion$ if [ -z "$UNKNOWN" ]; then
> echo "UNKNOWN" is undefined"
> fi
UNKNOWN is undefined
jhanasri@LAPTOP-1RN1M8E3:~/variable_expansion$ NAME="Jhanasri"
jhanasri@LAPTOP-1RN1M8E3:~/variable_expansion$ MESSAGE="Hello $NAME"
jhanasri@LAPTOP-1RN1M8E3:~/variable_expansion$ echo "$MESSAGE"
Hello Jhanasri

jhanasri@LAPTOP-1RN1M8E3:~$ mkdir variable_expansion
jhanasri@LAPTOP-1RN1M8E3:~$ cd variable_expansion
jhanasri@LAPTOP-1RN1M8E3:~/variable_expansion$ pwd
/home/jhanasri/variable_expansion
jhanasri@LAPTOP-1RN1M8E3:~/variable_expansion$ nano variable_expand.c
jhanasri@LAPTOP-1RN1M8E3:~/variable_expansion$ gcc variable_expand.c -o variable_expand
jhanasri@LAPTOP-1RN1M8E3:~/variable_expansion$ ls
variable_expand  variable_expand.c
jhanasri@LAPTOP-1RN1M8E3:~/variable_expansion$ ./variable_expand
===== VARIABLE EXPANSION PRACTICAL =====

Test 1: Basic variable expansion
Input : Hello $NAME
Output: Hello Jhanasri

Test 2: Multiple variables
Input : $NAME is studying $COURSE in $YEAR
Output: Jhanasri is studying OSSP in 2026

Test 3: Undefined variable
Undefined variable: UNKNOWN
Input : Value of $UNKNOWN
Output: Value of"

Test 4: Braced variable
Input : Student: ${NAME}
Output: Student: Jhanasri

Test 5: Nested variable
Input : $MESSAGE
First expansion: Welcome $NAME
Nested expansion: Welcome Jhanasri

===== EXPANSION TEST COMPLETED =====
jhanasri@LAPTOP-1RN1M8E3:~/variable_expansion$ █
```

Result: The variable expansion program was successfully implemented and executed. Variable references were detected and expanded correctly, including multiple, braced, undefined, and nested variables. The expansion logic produced the expected outputs for all test cases.

Observation: The program correctly replaced variables with their assigned values and handled undefined variables without errors. Nested variables were expanded successfully, and the token values were updated correctly during processing.

2 . To Identify Built-ins, Create Dispatch Table, Execute In-Process Commands, Handle Invalid Commands, Maintain State, Test Dispatch Logic

Aim : To identify shell built-in commands, create a dispatch table for those commands, execute built-in commands within the shell process, handle invalid commands, maintain shell state, and test the command-dispatch logic.

Program :

```
GNU nano 7.2
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>

#define MAX_INPUT 100
#define MAX_ARGS 10

/* Function declarations */
void builtin_pwd(char **args);
void builtin_cd(char **args);
void builtin_echo(char **args);
void builtin_exit(char **args);

/* Structure for dispatch table */
struct builtin_command
{
    char *name;
    void (*function)(char **args);
};

/* Dispatch table */
struct builtin_command builtins[] =
{
    {"pwd", builtin_pwd},
    {"cd", builtin_cd},
    {"echo", builtin_echo},
    {"exit", builtin_exit}
};

int builtin_count = 4;

/* pwd command */
void builtin_pwd(char **args)
{
    char cwd[1024];

    if (getcwd(cwd, sizeof(cwd)) != NULL)
    {
        printf("%s\n", cwd);
    }
    else
    {

```

```
{
    perror("pwd");
}

/* cd command */
void builtin_cd(char **args)
{
    if (args[1] == NULL)
    {
        printf("cd: missing directory\n");
        return;
    }

    if (chdir(args[1]) != 0)
    {
        perror("cd");
    }
}

/* echo command */
void builtin_echo(char **args)
{
    int i = 1;

    while (args[i] != NULL)
    {
        printf("%s", args[i]);

        if (args[i + 1] != NULL)
        {
            printf(" ");
        }

        i++;
    }

    printf("\n");
}

/* exit command */
void builtin_exit(char **args)
{
|
```

```
{
    printf("Exiting shell...\n");
    exit(0);
}

/* Find command in dispatch table */
int find_builtin(char *command)
{
    int i;

    for (i = 0; i < builtin_count; i++)
    {
        if (strcmp(command, builtins[i].name) == 0)
        {
            return i;
        }
    }

    return -1;
}

/* Main function */
int main()
{
    char input[MAX_INPUT];
    char *args[MAX_ARGS];
    char *token;
    int argc;
    int index;

    printf("==== BUILT-IN COMMAND SHELL =====\n");
    printf("Available commands: pwd, cd, echo, exit\n\n");

    while (1)
    {
        printf("myshell> ");

        if (fgets(input, sizeof(input), stdin) == NULL)
        {
            break;
        }

        /* Remove newline */
    }
```

```
        break;
    }

    /* Remove newline */
    input[strcspn(input, "\n")] = '\0';

    /* Ignore empty input */
    if (strlen(input) == 0)
    {
        continue;
    }

    /* Tokenize input */
    argc = 0;
    token = strtok(input, " ");

    while (token != NULL && argc < MAX_ARGS - 1)
    {
        args[argc++] = token;
        token = strtok(NULL, " ");
    }

    args[argc] = NULL;

    /* Find command in dispatch table */
    index = find_builtin(args[0]);

    if (index != -1)
    {
        printf("[Built-in command detected]\n");

        /* Execute command in current process */
        builtins[index].function(args);
    }
    else
    {
        printf("Invalid command: %s\n", args[0]);
    }
}

return 0;
}
```

^G Help
^X Exit

^O Write Out
^R Read File

^W Where Is
^\ **Replace**

^K Cut
^U Paste

Output :

```
sri@LAPTOP-1RN1M8E3:~$ mkdir builtin_practical
sri@LAPTOP-1RN1M8E3:~$ cd builtin_practical
sri@LAPTOP-1RN1M8E3:~/builtin_practical$ pwd
~/jhanasri/builtin_practical
sri@LAPTOP-1RN1M8E3:~/builtin_practical$ nano builtin_shell.c
sri@LAPTOP-1RN1M8E3:~/builtin_practical$ gcc builtin_shell.c -o builtin_shell
sri@LAPTOP-1RN1M8E3:~/builtin_practical$ ./builtin_shell
  BUILT-IN COMMANDS SHELL =====
  Available commands: pwd, cd, echo, exit

all> pwd
t-in command detected]
~/jhanasri/builtin_practical
all> echo Hello World
t-in command detected]
Hello World
all> echo OSSP Practical
t-in command detected]
Practical
all> pwd
t-in command detected]
~/jhanasri/builtin_practical
all> cd /tmp
t-in command detected]
all> pwd
t-in command detected]

all> hello
Invalid command: hello
all> xyz
Invalid command: xyz
all> cd
t-in command detected]
Missing directory
all> exit
t-in command detected]
Exiting shell...
sri@LAPTOP-1RN1M8E3:~/builtin_practical$
```

Result: The built-in shell program was successfully implemented and executed. Commands such as `pwd`, `cd`, `echo`, and `exit` were identified and executed within the shell process. Invalid commands were also detected and reported correctly.

Observation: The dispatch table correctly mapped commands to their corresponding built-in functions. The shell maintained its state during directory changes, handled invalid commands appropriately, and exited successfully when the `exit` command was used.